

Project Report: Deteccion de Imagenes IA vs Reales

1. Problem Statement

Este proyecto aborda un problema de clasificacion binaria en vision por computador: distinguir imagenes generadas por inteligencia artificial (clase IA) de imagenes reales (clase Real). El objetivo principal fue construir y comparar varios modelos de deep learning para estimar la etiqueta correcta en un conjunto de imagenes no vistas, maximizando la capacidad de generalizacion y manteniendo un balance razonable entre precision y recall.

La motivacion tecnica del problema es relevante en escenarios de verificacion de contenido digital, trazabilidad y analisis forense de imagenes. Desde la perspectiva de aprendizaje automatico, el reto radica en que ambas clases pueden compartir estructuras visuales similares, por lo que los modelos deben aprender patrones de alto nivel mas alla de rasgos superficiales.

2. Model Architecture

Se evaluaron cuatro arquitecturas:

1. **SimpleCNN**: red convolucional base disenada como baseline, con menor complejidad y costo computacional.
2. **LeNet**: arquitectura clasica de CNN adaptada al problema binario.
3. **VGG16 Fine-Tuned**: modelo preentrenado (transfer learning), ajustado para clasificacion binaria y posteriormente afinado.
4. **ResNet18 Fine-Tuned**: modelo residual preentrenado con etapa de transfer learning y etapa de fine-tuning en capas finales.

En todos los casos, la salida se ajusto para una tarea binaria con funcion de perdida BCE.

3. Training Procedure

El pipeline de entrenamiento siguio una estrategia uniforme para facilitar comparaciones justas:

- **Division del dataset:**
 - `train_test_split(test_size=0.2, seed=42)` sobre el conjunto completo.
 - sobre `train_val`, `train_test_split(test_size=0.125, seed=42)`.
 - proporciones finales aproximadas: **70% train / 10% val / 20% test**.
- **DataLoader:**
 - `batch_size = 64`
 - `num_workers = 0`
 - `pin_memory = device.type == 'cuda'`
- **Optimizacion:**
 - `nn.BCELoss()` para los modelos evaluados.
 - Adam con:
 - `lr = 1e-3` para modelos base (SimpleCNN y LeNet).
 - `lr = 1e-4` en etapa de transfer learning de VGG16/ResNet18.
 - `lr = 1e-5` en etapa de fine-tuning.
- **Regularizacion y control de entrenamiento:**
 - `EARLY_STOP_PATIENCE = 5`.
 - scheduler basado en validacion para reducir learning rate cuando la perdida de validacion no mejora.
 - guardado del mejor checkpoint por `val_loss`.
- **Epocas:**
 - los bloques principales usan `num_epochs = 5` en las fases documentadas.

Esta estrategia combina aprendizaje desde cero (baselines) y transfer learning (modelos profundos), permitiendo comparar capacidad representacional y costo de ajuste.

4. Validation Strategy

La validacion se realizo durante entrenamiento con un conjunto separado (10% del total), usando `val_loss` y `val_acc` para:

- monitorear estabilidad y sobreajuste,
- activar early stopping,

- seleccionar el mejor checkpoint.

La evaluacion final se hizo en test (20% del total), conjunto nunca visto durante entrenamiento ni ajuste de hiperparametros. Ademas de accuracy, se incluyeron reportes de clasificacion y matrices de confusion para analizar errores por clase.

5. Results

Las matrices de confusion finales (test) para los 4 modelos fueron:

- **SimpleCNN**: [[14885, 1352], [3931, 10374]]
- **LeNet**: [[15342, 895], [4488, 9817]]
- **VGG16 Fine-Tuned**: [[14793, 1444], [2167, 12138]]
- **ResNet18 Fine-Tuned**: [[15166, 1071], [2897, 11408]]

Con base en esas matrices (N = 30,542), se obtuvieron las siguientes metricas:

Modelo	Accuracy	Precision (IA)	Recall (IA)	F1 (IA)	Macro-F1
SimpleCNN	0.8270	0.7911	0.9167	0.8493	0.8232
LeNet	0.8238	0.7737	0.9449	0.8507	0.8178
VGG16 Fine-Tuned	0.8818	0.8722	0.9111	0.8912	0.8809
ResNet18 Fine-Tuned	0.8701	0.8396	0.9340	0.8843	0.8681

Resumen comparativo:

- El mejor desempeno global lo obtuvo **VGG16 Fine-Tuned** (mayor accuracy y macro-F1).
- **ResNet18 Fine-Tuned** quedo en segundo lugar, con muy buen recall en IA.
- Los modelos base (SimpleCNN y LeNet) mostraron buen recall para IA, pero menor precision por mayor tasa de falsos positivos.
-

6. Error Analysis

El analisis de errores revela un patron consistente:

1. Los modelos con menor capacidad (SimpleCNN y LeNet) tienden a **sobrepredicir la clase IA**, elevando falsos positivos (imagenes reales clasificadas como IA).
2. Los modelos con transfer learning reducen esos falsos positivos y mejoran el balance entre clases.
3. En ResNet18 Fine-Tuned, aunque el recall de IA es alto (0.9340), aun hay una cantidad considerable de reales etiquetadas como IA (FP = 2897), lo cual impacta precision.
4. VGG16 Fine-Tuned logra el mejor compromiso: menor error cruzado entre clases y mejor robustez global.

Posibles causas de error residual:

- similitud textural entre ciertas imagenes reales y sinteticas,
- ruido o variabilidad no controlada en el dataset,
- limite en la cantidad de epocas y en la profundidad del ajuste fino.
-

Conclusion

La comparacion muestra que **transfer learning + fine-tuning** supera claramente a las CNN entrenadas desde cero para este problema. El modelo recomendado por desempeno es **VGG16 Fine-Tuned**. Como trabajo futuro, se propone ampliar augmentations, explorar calibracion de umbral de decision y realizar ajuste de hiperparametros mas exhaustivo para reducir aun mas los errores de clase cruzada.